

Make It Count: Text-to-Image Generation with an Accurate Number of Objects

Lital Binyamin¹ Yoad Tewel^{2,3} Hilit Segev¹ Eran Hirsch¹ Royi Rassin¹ Gal Chechik^{1,2}

¹Bar-Ilan University ²NVIDIA ³Tel-Aviv University



Figure 1. **CountGen** generates the correct number of objects specified in the input prompt while maintaining a natural layout that aligns with the prompt.

Abstract

Despite the unprecedented success of text-to-image diffusion models, controlling the number of depicted objects using text is surprisingly hard. This is important for various applications from technical documents, to children’s books to illustrating cooking recipes. Generating object-correct counts is fundamentally challenging because the generative model needs to keep a sense of separate identity for every instance of the object, even if several objects look identical or overlap, and then carry out a global computation implicitly during generation. It is still unknown if such representations exist. To address count-correct generation, we first identify features within the diffusion model that can carry the object identity information. We then use them to separate and count instances of objects during the denoising process and detect over-generation and under-generation. We fix the latter by training a model that predicts both the shape and location of a missing object, based on the layout of existing ones, and show how it can be used to guide denoising with correct object count. Our approach, **CountGen**, does not depend on external source to determine object layout, but rather uses the prior from the

diffusion model itself, creating prompt-dependent and seed-dependent layouts. Evaluated on two benchmark datasets, we find that **CountGen** strongly outperforms the count-accuracy of existing baselines. Project page: <https://make-it-count-paper.github.io/>

1. Introduction

Text-to-image diffusion models provide an accessible way to control the generation of visual content. A major failure mode is their inability to count, that is, they often fail to generate the correct number of items in response to text prompts. For instance, when asked to generate an image of Goldilocks and the three bears, models may generate only two bears (Fig. 1). Counting failures are particularly frustrating: The accuracy is surprisingly low, and mistakes are often obvious for people to detect.

To illustrate the difficulty of the problem, consider some naive attempts to work around it. First, one can manually design layouts per count, to determine the spatial organization and the number of objects, then provide it as a conditioning signal to a generative model [9]. This approach

would fail to generate prompt-dependent layouts, which is highly desirable. One could also try asking large vision-language models to propose layouts (e.g., [7, 12]), but these approaches do not use the visual prior information that text-to-image models already collected, and, as we show below, their performance is rather poor for the counting task.

Why is it so hard for diffusion models to count while they generate? First, counting objects requires that models capture “objectness” – the high-level coherent representation of something being a separate entity, even if surrounded by other similar entities. Capturing objectness is by itself a hard task in image understanding [1, 19], and long studied in cognitive psychology [30]. It is currently not known to what extent diffusion models represent objectness of entities they generate. A second main challenge is that text-to-image diffusion models struggle with controlling spatial layout just from text. Producing a correct number of objects requires obeying a global and complex spatial relation between entities in an image [6, 8].

To address the problem of accurate count generation we describe several new contributions, which together form our method *CountGen*. First, we analyze the representations of the self-attention layers in SDXL [25], and identify features that capture objectness and instance identity. We then use these features to develop ways to detect instances of objects during the denoising process, find their spatial layout and count them. Specifically, we localize the features that correspond to objects using the cross-attention maps and cluster them to form object instance segmentation. Importantly, we do not have to wait for an image to be fully generated, and we can accurately count the number of objects already at an intermediate step of the denoising process.

Given this new capability to count the number of objects being generated during the denoising process, we further develop methods to correct generation when the count does not match the prompt. First, we train a layout-modification network we call ReLayout. It takes a spatial layout of k objects and generates a similar spatial layout with one more instance of an object added in a natural location for the input layout. For example, given a row of five kittens sitting on a branch, it learned to add a sixth kitten to the same row. This model is trained using image-pair samples generated by the diffusion model itself. Finally, we show how to use the new layouts in a new test-time-optimization procedure.

We evaluate CountGen on text prompts from the T2I-CompBench [16] which includes prompts with numbers. CountGen greatly improves accuracy, as evident by human evaluation experiments, from 29% accuracy for SDXL to 48% by our method. It also improves over all other baseline methods including large commercial models like the recent DALL-E 3 [3]. To support future work in this field, we design and release a dataset that can be evaluated automatically. Specifically, we release CoCoCount, a set of

prompts based on COCO classes [21], which can easily be evaluated using COCO-trained object detectors, like YOLO [35]. CountGen also significantly improves over all baseline methods on CoCoCount, importantly from 26% accuracy for SDXL to 52% by our method.

In summary, this paper makes the following new contributions (1) We identify novel features that represent objectness and instance identity in SDXL [25]. (2) We design an inference-time optimization to guide SDXL to generate an accurate number of instances for an object. (3) We describe a learning approach to automatically modify layouts to add a new instance of an object while preserving the structure of the scene. (4) We achieve state-of-the-art results in count-accurate generation.

2. Related Work

Generating images with accurate object count. Numerous papers noted that text-to-image diffusion models often fail to produce images that accurately match text prompts, especially when these prompts specify an exact number of objects [2, 6, 8, 11, 12, 17, 20, 23, 26, 27, 32, 37, 41]. Various efforts were made to improve the accuracy of these models. Most relevant to our work, Kang et al. [17] proposed a classifier-guidance approach to improve object count accuracy. The method “counts” instances at each diffusion step using a pretrained counting network and adjusts the denoising process using gradient guidance. However, it requires using an additional U-Net in *every* denoising step. Additionally, Zafar et al. [40] proposed a token optimization method; however, it requires optimizing a new token over multiple steps before using it for inference.

An important line of work suggests breaking the generation process into two steps: (1) Text-to-layout - setting a spatial location for every object instance; and (2) Layout-to-image - generating an image with the correct object count using the given layout. **Text-to-Layout:** Several studies used large language models (LLMs) to propose spatial layouts [7, 12, 13, 24]. LayoutGPT [12] injects visual commonsense into the LLM prompt which enables it to generate desirable layouts. Gani et al. [13] suggest decomposing complex prompts into smaller prompts before injecting them into the LLM. **Layout-to-image:** Providing a predefined layout with the exact number of subjects helps ensure that the generated images reflect the intended count [7, 39]. Bounded Attention [8] addresses this challenge by channeling attention to bounding boxes corresponding to object instances. However, this approach requires users to manually provide the bounding boxes for all the instances. In contrast to these separate-step approaches, CountGen addresses the two steps of count-accurate generation. It first corrects the layout that emerges during generation so it contains the correct number of instances and then uses a novel test-time optimization method to generate a count-accurate image.

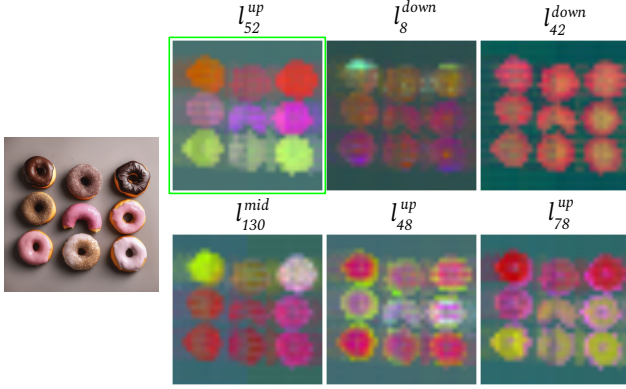


Figure 2. **PCA visualization.** To explore the notion of objectness inside SDXL latent space, we visualize dimension-reduced self-attention feature maps from various layers across the network at timestep $t = 500$. We notice that although most layers do not exhibit a clear separation of objects, layer l_{52}^{up} displays a robust separation indicated by different object instances having distinct colors. Visualization across different timesteps is shown in the appendix, Fig. 14.

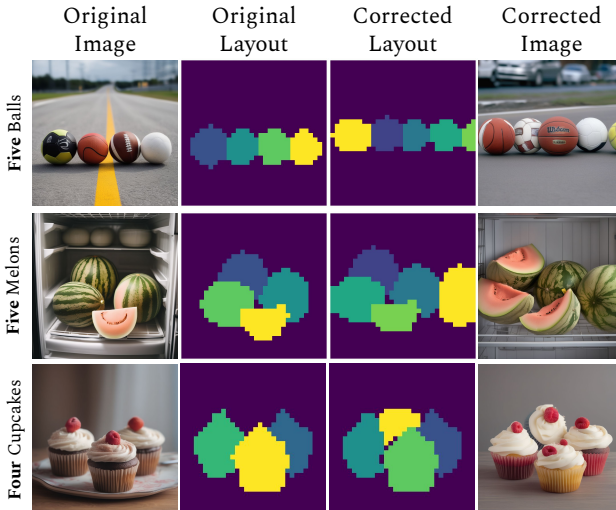


Figure 3. **Correcting under-generation.** We show examples for the ReLayout correction of cases where SDXL generates less objects than specified in the prompt. It is evident that the generated layouts are natural and obey the same composition of the original generation, with the correct number of objects.

Controlling text-to-image models through attention-based loss. To address the issue of object neglect—when objects mentioned in a text prompt fail to appear in the generated image—Chefer et al. [6] developed a novel loss function that ensures all objects in the prompt are reflected in the cross-attention maps used during image generation. Rassin et al. [27] tackled the challenge of incorrect attribute association by designing a loss function that binds the cross-attention maps of subjects and their attributes more effec-

tively. Inspired by these advancements, CountGen includes a novel cross-attention maps loss function designed to ensure the generation adheres closely to the input layout.

3. Our Approach: CountGen

Our method, CountGen, aims to enhance text-conditioned image generators to accurately produce the intended number of objects for complex input prompts. Our methodology involves a two-step process: initially, we generate a *natural* layout that specifies where and how objects should appear in the image (Sec. 3.2). That layout is based on a layout that emerges naturally from the text-conditioned generation (Sec. 3.1). At the second step (Sec. 3.3), we use this layout as a blueprint to generate the final image.

3.1. Discover Layout During Early Generation

To count object instances during generation, one must first find an internal representation that captures the separate identity of different object instances. It is not known if this representation exists in diffusion models such as SDXL. We now discuss this representation and then show how we can detect the layout of objects during early generation.

An emerging instance-identity representation in SDXL.

We begin by exploring the notion of ‘objectness’ in SDXL. While previous work [6, 14, 33] utilized the cross-attention mechanism to localize *objects of a given class* in generated images, little research has been conducted on whether the model encodes information about *object instances* and how to distinguish between different instances of an object.

We tackle this problem by conducting a feature analysis across different timesteps and layers, as described in Sec. 6 and Appendix C.5. We discover that l_{52}^{up} outperforms all others at different timesteps. Fig. 2 illustrates this analysis using PCA visualization of self-attention features from various layers across SDXL at timestep $t = 500$, which shows the most robust instance representation (See Appendix Fig. 14). Layer l_{52}^{up} generates different features for different instances of the same object, with each instance having its distinct color.

Identifying object instances. Building on the findings of Hertz et al. [14], which show that cross-attention maps can pinpoint a token’s position in a generated image, we create a *foreground mask* for each object described in the prompt. By contrasting these foreground masks, derived from the cross-attention, with the self-attention features, we effectively segregate pixels associated with objects from those belonging to the background. Subsequently, we cluster the object-associated pixels from the self-attention map into distinct masks for each object. This approach allows us to

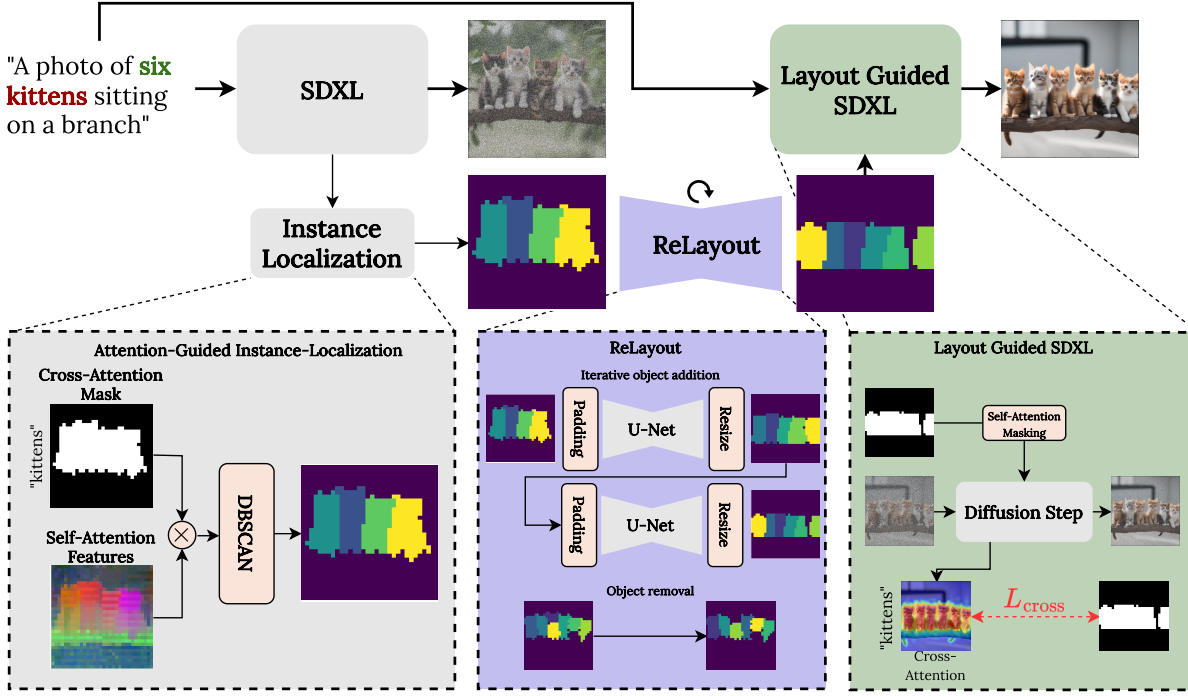


Figure 4. **Architecture outline.** Given a prompt that includes a quantity, we begin generating a corresponding image using pretrained SDXL until timestep $t = 500$. We then perform **Instance Localization**, where we combine cross-attention maps corresponding with the object, and self-attention features extracted at timestep t to generate object clusters for each generated object. Then we apply **ReLayout**, which generates an object layout with the correct number of instances, while preserving the composition of the extracted layout. Finally, we perform **Layout Guided** generation, which applies an inference time optimization based on the layout through cross-attention loss L_{cross} and self-attention masking.

refine our object representations and enhance the accuracy of the generated layouts.

Formally, let $A_{l,t}^{\text{self}}, A_{l,t}^{\text{cross}}$ represent the self-attention and cross-attention maps, respectively, for layer l at timestep t within our diffusion network. We aggregate cross-attention maps associated with the tokens corresponding to the objects specified in the input prompt. We then use these cross-attention maps to extract a foreground mask M based on dynamic thresholding $M = \text{Otsu}(A_{l,t}^{\text{cross}})$, where Otsu applies the Otsu thresholding method [22, 34] to determine foreground (object) pixels. We define $p_k \subseteq A_{l,t}^{\text{self}}$ as the set of features from the self-attention map that are identified as foreground by mask M . We then cluster these patches: $\text{Clusters} = \text{DBSCAN}(p_k, \epsilon)$, where $\text{DBSCAN}(\cdot, \epsilon)$ is the DBSCAN [10] clustering with a dynamic parameter ϵ . Finally, the initial layout L is created by grouping the object clusters: $L = \bigcup_{C \in \text{Clusters}} C$.

At the end of this process, we obtain a set of masks, one for each object being generated. This is illustrated in Fig. 4, left gray box.

3.2. ReLayout: Correcting the Number of Objects

We now introduce our layout-correction component, *ReLayout*, which **preserves the overall scene composition** while correcting the number of objects. For example, Fig. 4 depicts an image generated using the prompt “a photo of six cats”, but only four cats were generated. Our ReLayout generates a new layout with the correct number of instances while keeping the overall composition of the kittens sitting in a row. More examples are shown in Fig. 3.

The input to the ReLayout is an object-layout described in Sec. 3.1, from which we initially infer the number of generated instances. Next, our ReLayout component takes one of two corrective actions based on the discrepancy between the generated and expected counts. In cases of **over-generation**, where more instances were generated than requested, ReLayout deterministically removes the smallest instances to achieve the desired cluster count. We find that this simple strategy produces appealing results. In cases of **under-generation**, a more intricate challenge arises: the ReLayout must insert new instances to the scene in a way that preserves the original scene structure. This process involves a sophisticated understanding of different object lay-

outs—like the stark contrast between linearly arrayed *bottles* and the clustered arrangement of *elephants*—to seamlessly augment the layout. In Sec. 3.2.1, we detail our approach for handling under-generation. In cases where the number of instances is correct, the ReLayout maintains the initial layout.

3.2.1. Handling Under-Generation

To address under-generation issues, we train a U-Net model to predict a new layout, represented as a multi-channel mask, from an existing layout. In practice, each forward pass of the U-Net generates a mask with an additional instance. This process is applied in iterations until the mask reflects the correct number of instances. In what follows we provide detailed information on the architecture and training of our U-Net model.

Creating a training dataset. To train our ReLayout U-Net, we need a dataset of layout pairs with k and $k + 1$ objects, that maintain the same scene composition. We begin with the empirical observation that slight variations in the object count specified in the prompt—while keeping the starting noise and the rest of the prompt consistent—typically results in images with similar layouts, as shown in Appendix Fig. 9. This consistency is crucial as it allows us to generate a training dataset of layout pairs where each pair has a similar object composition, differing by only one object, thereby preserving the overall scene structure.

Following this observation, we generate a set of $\sim 10K$ pairs of images of I_k and I_{k+1} , where each pair consists of images that differ by only one in the number of objects depicted. Each pair is generated with random fixed seeds and prompts that fit the same template, such as “a photo of two cats” versus “a photo of three cats”. To confirm that each image pair accurately represents an k and $k + 1$ object scenario, we extract object masks M_k and M_{k+1} as described in Sec. 3.1, and verify the object count in one image is exactly one more than in its paired image. Overall, the final dataset for training consists of pairs of binary masks (M_k, M_{k+1}) , representing the U-Net task of learning to generate a mask with $k + 1$ objects from a mask with k objects.

Matching objects. To train the U-Net, we need to establish a correspondence between each object i in M_k to its new position in M_{k+1} . We aim to find a matching that minimizes the shift in objects positions. We use the Hungarian algorithm [18] to find the optimal matching. More details in Appendix C.1.

Training the U-Net module. We trained the U-Net architecture by adapting it to handle 9 input channels – corresponding to the source tensor $M_k \in \{0, 1\}^{W \times H \times k}$ with k

objects, and output 10 channels – for the target tensor with $k + 1$ objects, to support counts up to 10. We optimized the U-Net parameters using two loss functions: (1) A Dice loss [31] between a predicted masks $\hat{M}_{k+1}$ and the target masks M_{k+1} of that object; and (2) Mask-to-mask overlap loss, designed to reduce the overlap between output masks of different instances. Specifically, this was computed as $1 - L_{Dice}$ between all pairs of predicted masks $\hat{M}_{k+1}^i, \hat{M}_{k+1}^j$.

$$\mathcal{L} = \mathcal{L}_{Dice} + \lambda \mathcal{L}_{overlap} \quad (1)$$

with λ being a weighing hyperparameter. Detailed definitions are provided in Appendix C.2, and evaluation of ReLayout is in Appendix C.3.

Inference. At inference time, as a pre-processing step, we first add padding to input masks. After each iteration, we gradually and consistently increase the padding size around the original masks. This operation is beneficial when we need to add a large number of objects, as it creates a “zoom-out” effect, making space for new objects.

We also slightly erode instance masks after the ReLayout, to improve separation of contacting objects.

3.3. CountGen: Layout-Based Image Generation

Provided with correct object mask layouts (Sec. 3.2), our goal is to guide the image generation process to adhere to the input layout. Given a mask for each object in the desired layout, we apply an inference time optimization to match the layout in the generated image. To optimize object layouts at inference time, we propose a dual approach: object layout loss to encourage object creation in the foreground, i.e. pixels within the object masks, and self-attention masking to prevent object generation in the background.

Object layout loss. Consider the optimization of object placement within layouts using a weighted binary cross-entropy loss. Given c , the aggregated cross-attention scores, and m , a binary mask denoting object presence (foreground), the weighted binary cross-entropy loss is computed pixel-wise and is defined as follows:

$$L(c, m) = - \sum_i w_i (m_i \log c_i + (1 - m_i) \log(1 - c_i)),$$

where c_i is the cross-attention score at pixel i , m_i is the value of the binary mask at pixel i , and w_i is the weight assigned to each pixel i where $w_i = 10$ if $m_i = 1$, otherwise $w_i = 1$. During the SDXL generation process, each step takes a noised latent X_t as input. For the first 25 generation steps, we propagate gradients from the object layout loss to X_t , updating it to reduce the loss.

Self-attention masking. The object-layout loss encourages objects to be generated in the foreground, but when applied on itself, generated objects may appear outside the object masks. To address this, we mask the self-attention connections between pixels in the background to pixels in the foreground. By disrupting these links, we stop the information flow from the objects to the rest of the image and prevent the model from forming objects in the background. Formally, at layer l and timestep t , the masked self-attention $S_t^{*(l)}$ is defined as:

$$S_t^{*(l)}[i, j] = \begin{cases} 0 & \text{if } i \in \mathcal{B}^{(l)} \text{ and } j \in \mathcal{F}^{(l)}, \\ S_t^{(l)}[i, j] & \text{otherwise.} \end{cases}$$

where i and j are pixels indices, $\mathcal{B}^{(l)}$ and $\mathcal{F}^{(l)}$ represents the set of pixels belonging to the background and the foreground respectively, and $S_t^{(l)}$ is the self-attention map at layer l and timestep t . We discuss implementation details and computational efficiency in Appendix A.

4. Experiments

Compared methods. We compare CountGen against seven baseline methods: (1) **SDXL** [25]; (2) **Repeated Object**: SDXL, with a modified prompt, where an object is repeated in the prompt the number of times it is required to generate, as in replacing “three cats” with “a cat and a cat and a cat”. This is a naive approach that parallels prompts like “A cat and a dog”. (3) **Reason Out Your Layout**: [7] uses GPT-3.5 [4] to generate layouts then trained an adapter to integrate it to SD-1.4 [28]; (4) **DALL-E 3** [3]; (5) **Random masks + BoundedAttn**: generate a layout with the correct amount of clusters placed randomly in the image and apply a layout-guidance generation method on top; (6) **Counting Guidance** [17]: boost generation of SD with a counting network; (7) **RPG** [38]: generates the layout using GPT-4 and then uses SDXL. (8) **Token Optimization** [40]: optimizes the embeddings of an added counting token.

Full details on how we used these baselines are given in Appendix B.3. We also compared our layout-to-image phase, CountGen-Image, described in section Sec. 3.3 with Bounded Attention [8].

Datasets. We evaluate our method and the baselines using two datasets. (1) **T2I-Compbench-Count**. A subset of T2I-Compbench [16], which is a benchmark for open-world compositional text-to-image generation. This subset specifically includes 218 prompts that specify a single object and its number (between 2 to 10). (2) **CoCoCount (ours)**. We generate a dataset with automatic evaluation. Specifically, we sample classes from COCO, which are more favorable to accurate and automatic detection by methods, like YOLOv9 [35]. We design simple prompts around these classes, with

a number between 2 and 10. In total, there are 200 prompts with various classes, numbers and scenes. See full details in Appendix E.

Count accuracy evaluation. We evaluate the results of CountGen and the baselines using human and automatic evaluation method, which is standardized and reproducible. In both settings, we seek to identify if the number of instances generated by the object matches the prompt.

Human evaluation. We quantified the count-accuracy of our method and baselines using human raters. Raters were asked for every image: (1) Is the object in the image?; (2) Are its instances well-formed?; (3) How many instances of the object are in the image? If the answer to question (1) or (2) is “no”, then we do not ask question (3). We provide details on the platform, rater selection and pay, and screenshots of the task in Appendix F.1.

Automatic evaluation. For automatic evaluation, we use the YOLOv9 model [35] with its default settings, as it represents the current state-of-the-art in the YOLO object detection benchmarks. To extract the number of objects in the image, we simply count the number of detected bounding-boxes corresponding to the target object.

Image quality evaluation. Forcing the diffusion model to obey the count in the text prompt is inevitably expected to reduce the naturalness and visual appeal of generated images, simply because more constraints are added. This effect has been observed in other studies using test-time optimization [6, 27]. We evaluate the image quality of CountGen by presenting human raters with two images, by CountGen and SDXL, and asking them to select whether one image is more natural and well-formed than the other or to indicate that both images are equally good.

Table 1. Generated count accuracy. Values are the percent of generated images that have the correct number of objects, for CoCoCount and T2I-Compbench-Count.

Model	CoCoCount		Comp-bench
	YOLOv9 Accu- racy	Human Accu- racy	Human Accu- racy
SDXL	28	26	29
Repeated Object	17	18	14
Reason Layout	21	26	15
DALL-E 3	25	38	36
Random+BoundedAttn	29	30	35
Counting Guidance	21	22	22
RPG	21	28	25
Token Optimization	34	36	31
CountGen (ours)	50	52	48

5. Results

Quantitative results. Tab. 1 compares CountGen with competing baselines, demonstrating its significant improvement in both CoCoCount and T2I-compbench-Count. Fig. 6 shows CountGen generally outperforms all baselines, except in two and three instances where DALL-E 3 and Token Optimization slightly outperform, respectively. This performance of DALL-E 3 may be attributed to its larger size and large training data compared to SDXL, our base model. Regarding image quality, among 200 comparisons, SDXL was preferred in only 23 cases, indicating minimal quality loss in CountGen. Additional quality metrics are detailed in Sec. C.7. The confusion matrix in Appendix (Fig. 20) reflects the human evaluation of CountGen. Additional Appendix analyzes include run-time comparisons across baselines (Fig. 7), a high object count stress test (Sec. C.6), semantic and spatial evaluations (Sec. C.8), and an extension of CoCoCount to test the robustness of CountGen (Sec. C.10).

Qualitative results. Fig. 5 shows examples of prompts and the images generated by various methods. In contrast to other methods, CountGen consistently generates the correct object number.

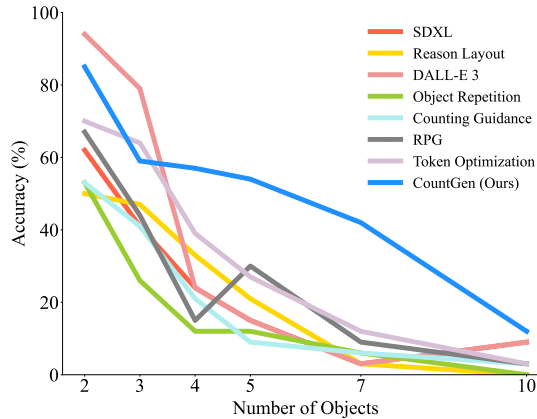


Figure 6. **Accuracy, as a function of the number of generated objects.** Accuracy evaluated by human raters, over the set of 200 evaluation images. CountGen (blue) outperforms all methods for $n > 3$, and is on par with DALL-E 3 for 2 and 3 objects.

6. Ablation Study

Contribution of CountGen-Layout and CountGen-Image. Tab. 2 quantifies the contribution of each of these components to the overall accuracy, by replacing it with a baseline alternative. Compared with a baseline (Random Masks + Bounded Attention) our first phase CountGen-Layout improves accuracy measured by people by 14%

Table 2. Model components Accuracy (%).

		CoCoCount		Compbench
Text → Layout	Layout → Image	YOLOv9 Acc.	Human Acc.	Human Acc.
CountGen	CountGen	50	52	48
CountGen	B-Attn	40	42	40
Random	CountGen	37	44	42
Random	B-Attn	29	30	35

Table 3. CountGen-Layout components. Error bars represent standard error across 200 images.

Method	Precision	Recall	IOU
CountGen	59±3.1	82±2.5	52±1.2
- SA masking	48±3.1	81±2.7	51±1.5
- Layout loss	49±2.9	64±2.5	36±1.4

(from 30 to 44), and our second phase CountGen-Image by 12%. Together, the two components add up to improve accuracy by 21 points. We also expand the comparison to other layout-to-image methods in Appendix C.9.

Layout guided generation ablation study. The second phase of our method, CountGen-Image, consists of two components: self-attention masking and object layout loss, as described at Sec. 3.3. To evaluate the contribution of each component, we deactivate it and compare the results. In Appendix Fig. 15, we qualitatively observe that removing the layout loss leads to the objects scattering in the image, not constrained by the required mask. When removing the self-attention masking the objects tend to obey the mask unwanted object instances occur in the background.

We confirm these observations quantitatively in Tab. 3, where we evaluate the adherence of the generated image to the input mask. We use YOLOv9 to detect the bounding boxes of generated objects and compare them to the input mask using three metrics: *Precision* is the percentage of bounding boxes that highly overlap ($IOU > 0.6$) the mask (union of all object masks), *Recall* is the percentage of mask pixels that are covered by bounding boxes, and *IOU* is measured between the boxes and the mask. Our findings align with the qualitative observation: removing the self-attention masking leads to a worse precision score, meaning objects are generated in the background. Removing the layout loss leads to low recall and IOU, meaning poor adherence to the mask. CountGen-Image, employing both components, achieves balanced results by generating objects in accordance with the mask. Overall, these results emphasize the critical roles that both components in ensuring accurate adherence to the input mask.

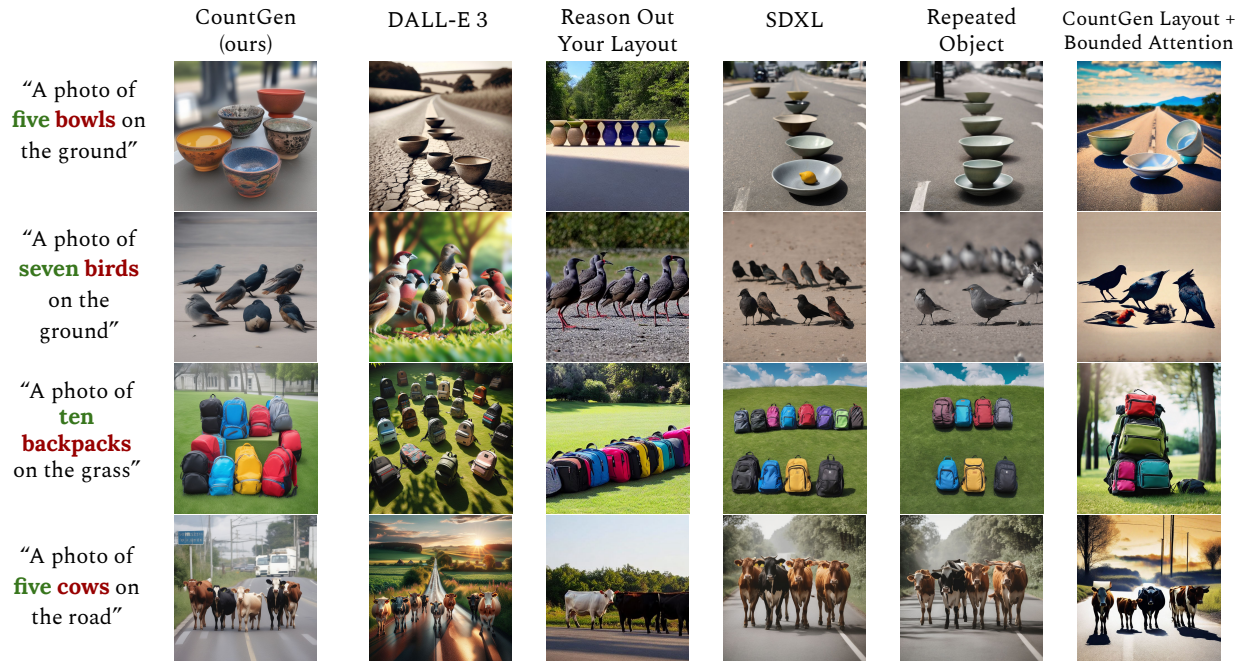


Figure 5. **Qualitative comparisons.** We evaluated CountGen against DALL-E 3, Reason Out Your Layout, SDXL, Repeated Object SDXL and CountGen Layout + Bounded Attention. Our method successfully generates the correct number of objects, while other methods struggle in some or all of the examples. Additional results are shown in the supplemental material (Fig. 22, 24).

Pipeline analysis. We identified three primary sources of failure within our pipeline: **Instance Localization**, **ReLayout-out** failures and **Layout Guidance**. We noticed that over-generation occurred mostly for target count k bigger than 5, whereas layout-guidance issues are more frequent with target counts ≤ 5 . See Appendix C.4 for more details.

Sensitivity analysis of instance-level features. To quantitatively evaluate the performance of our instance-localization step we compare the bounding box predictions extracted from our method’s instance localization masks to ground truth bounding boxes. The time and layer that we selected on set-aside data, generalize well to the test data, and out-perform other choices. Further details on the instance-level features are provided in Appendix C.5.

7. Limitations

Occasionally, our optimization (Sec. 3.3) results in multiple instances of an object in an area intended for just one by the layout. In other cases CountGen generates plain backgrounds compared to SDXL (Fig. 8 in Appendix). In addition, the scope of our experiments may seem narrow, since we focus on generating scenes with up to 10 instances and a single object per prompt. Nevertheless, we have shown in Sec. 5 that even this setup is highly challenging to contemporary models, especially as the number of instances required to generate grows, as evident by the massive drop in

performance, even for DALL-E 3 (see Fig. 6).

8. Conclusions

Generating images with the correct number of objects is challenging, as it demands models not only to grasp “objectness” and adhere to global spatial constraints but also to produce coherent natural images. Existing text-to-image diffusion models struggle with this, particularly when generating more than three objects (Tab. 1, Fig. 6).

Our CountGen approach takes three pivotal steps to tackle this issue. Initially, we extracted a concept of objectness from the self-attention layers of a diffusion model. Next, we trained a U-Net to adjust the number of object instances in layouts while maintaining structural integrity. Finally, we developed a layout-guidance optimization to generate images from these adjusted layouts. This strategy nearly doubled the counting accuracy from 26% with standard SDXL to 52% with our enhanced SDXL.

We expect the lessons learned from this method, specifically the features that represent objectness and the process of learning to automatically fix a layout, to become useful in other problems. Future work, outlined in Appendix D, include expanding CountGen to manage multiple objects and adapting the approach to different architectures.

9. Acknowledgments

We thank Yuval Atzmon for useful discussions and for providing feedback on an earlier version of this manuscript. This study was supported by an equipment grant from the Israel Science Foundation (ISF 2332/18), and by a grant from the National committee of budgeting (VATAT).

References

- [1] Bogdan Alexe, Thomas Deselaers, and Vittorio Ferrari. Measuring the objectness of image windows. *IEEE transactions on pattern analysis and machine intelligence*, 34(11):2189–2202, 2012. 2
- [2] Barak Battash, Amit Rozner, Lior Wolf, and Ofir Lindenbaum. Obtaining favorable layouts for multiple object generation, 2024. 2
- [3] James Betker, Gabriel Goh, Li Jing, Tim Brooks, Jianfeng Wang, Linjie Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, et al. Improving image generation with better captions. *Computer Science*. <https://cdn.openai.com/papers/dall-e-3.pdf>, 2(3):8, 2023. 2, 6, 12
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. 6
- [5] Mateusz Buda, Ashirbani Saha, and Maciej A Mazurowski. Association of genomic subtypes of lower-grade gliomas with shape features automatically extracted by a deep learning algorithm. *Computers in Biology and Medicine*, 109, 2019. 11
- [6] Hila Chefer, Yuval Alaluf, Yael Vinker, Lior Wolf, and Daniel Cohen-Or. Attend-and-excite: Attention-based semantic guidance for text-to-image diffusion models. *ACM Transactions on Graphics (TOG)*, 42(4):1–10, 2023. 2, 3, 6, 11
- [7] Xiaohui Chen, Yongfei Liu, Yingxiang Yang, Jianbo Yuan, Quanzeng You, Li-Ping Liu, and Hongxia Yang. Reason out your layout: Evoking the layout master from large language models for text-to-image synthesis, 2023. 2, 6, 11
- [8] Omer Dahary, Or Patashnik, Kfir Aberman, and Daniel Cohen-Or. Be yourself: Bounded attention for multi-subject text-to-image generation, 2024. 2, 6, 12
- [9] Omer Dahary, Or Patashnik, Kfir Aberman, and Daniel Cohen-Or. Be yourself: Bounded attention for multi-subject text-to-image generation. *arXiv preprint arXiv:2403.16990*, 2024. 1
- [10] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, page 226–231. AAAI Press, 1996. 4
- [11] Ying Fan, Olivia Watkins, Yuqing Du, Hao Liu, Moonkyung Ryu, Craig Boutilier, Pieter Abbeel, Mohammad Ghavamzadeh, Kangwook Lee, and Kimin Lee. Reinforcement learning for fine-tuning text-to-image diffusion models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. 2
- [12] Weixi Feng, Wanrong Zhu, Tsu-Jui Fu, Varun Jampani, Arjun Reddy Akula, Xuehai He, S Basu, Xin Eric Wang, and William Yang Wang. LayoutGPT: Compositional visual planning and generation with large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. 2
- [13] Hanan Gani, Shariq Farooq Bhat, Muzammal Naseer, Salman Khan, and Peter Wonka. LLM blueprint: Enabling text-to-image generation with complex and detailed prompts. In *The Twelfth International Conference on Learning Representations*, 2024. 2
- [14] Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-or. Prompt-to-prompt image editing with cross-attention control. In *The Eleventh International Conference on Learning Representations*, 2023. 3
- [15] Matthew Honnibal and Ines Montani. spacy 2: Natural language understanding with bloom embeddings, convolutional neural networks and incremental parsing. *To appear*, 7(1): 411–420, 2017. 11
- [16] Kaiyi Huang, Kaiyue Sun, Enze Xie, Zhenguo Li, and Xihui Liu. T2i-compbench: A comprehensive benchmark for open-world compositional text-to-image generation, 2023. 2, 6
- [17] Wonjun Kang, Kevin Galim, and Hyung Il Koo. Counting guidance for high fidelity text-to-image synthesis, 2023. 2, 6, 12
- [18] H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955. 5
- [19] Weicheng Kuo, Bharath Hariharan, and Jitendra Malik. Deepbox: Learning objectness with convolutional networks. In *Proceedings of the IEEE international conference on computer vision*, pages 2479–2487, 2015. 2
- [20] Kimin Lee, Hao Liu, Moonkyung Ryu, Olivia Watkins, Yuqing Du, Craig Boutilier, Pieter Abbeel, Mohammad Ghavamzadeh, and Shixiang Shane Gu. Aligning text-to-image models using human feedback, 2023. 2
- [21] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014. 2, 15
- [22] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979. 4
- [23] Roni Paiss, Ariel Ephrat, Omer Tov, Shiran Zada, Inbar Mosseri, Michal Irani, and Tali Dekel. Teaching clip to count to ten. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3170–3180, 2023. 2
- [24] Quynh Phung, Songwei Ge, and Jia-Bin Huang. Grounded text-to-image synthesis with attention refocusing, 2023. 2

- [25] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. Sdxl: Improving latent diffusion models for high-resolution image synthesis, 2023. [2](#), [6](#), [11](#)
- [26] Royi Rassin, Shauli Ravfogel, and Yoav Goldberg. DALLÉ-2 is seeing double: Flaws in word-to-concept mapping in Text2Image models. In *Proceedings of the Fifth Black-boxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, pages 335–345, Abu Dhabi, United Arab Emirates (Hybrid), 2022. Association for Computational Linguistics. [2](#)
- [27] Royi Rassin, Eran Hirsch, Daniel Glickman, Shauli Ravfogel, Yoav Goldberg, and Gal Chechik. Linguistic binding in diffusion models: Enhancing attribute correspondence through attention map alignment. *Advances in Neural Information Processing Systems*, 36, 2024. [2](#), [3](#), [6](#), [11](#)
- [28] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022. [6](#)
- [29] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. Laion-5b: An open large-scale dataset for training next generation image-text models, 2022. [13](#)
- [30] Elizabeth S Spelke. Principles of object perception. *Cognitive science*, 14(1):29–56, 1990. [2](#)
- [31] Carole H Sudre, Wenqi Li, Tom Vercauteren, Sebastien Ourselin, and M Jorge Cardoso. Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations. In *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support: Third International Workshop, DLMIA 2017, and 7th International Workshop, ML-CDS 2017, Held in Conjunction with MICCAI 2017, Québec City, QC, Canada, September 14, Proceedings 3*, pages 240–248. Springer, 2017. [5](#)
- [32] Jiao Sun, Deqing Fu, Yushi Hu, Su Wang, Royi Rassin, Da-Cheng Juan, Dana Alon, Charles Herrmann, Sjoerd van Steenkiste, Ranjay Krishna, and Cyrus Rashtchian. Dreamsync: Aligning text-to-image generation with image understanding feedback, 2023. [2](#)
- [33] Yoad Tewel, Rinon Gal, Gal Chechik, and Yuval Atzmon. Key-locked rank one editing for text-to-image personalization, 2023. [3](#)
- [34] Yoad Tewel, Omri Kaduri, Rinon Gal, Yoni Kasten, Lior Wolf, Gal Chechik, and Yuval Atzmon. Training-free consistent text-to-image generation. *arXiv preprint arXiv:2402.03286*, 2024. [4](#)
- [35] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9: Learning what you want to learn using programmable gradient information. *arXiv preprint arXiv:2402.13616*, 2024. [2](#), [6](#)
- [36] Xudong Wang, Trevor Darrell, Sai Saketh Rambhatla, Rohit Girdhar, and Ishan Misra. Instancediffusion: Instance-level control for image generation, 2024. [14](#)
- [37] Song Wen, Guian Fang, Renrui Zhang, Peng Gao, Hao Dong, and Dimitris N. Metaxas. Improving compositional text-to-image generation with large vision-language models, 2024. [2](#)
- [38] Ling Yang, Zhaochen Yu, Chenlin Meng, Minkai Xu, Stefano Ermon, and Bin Cui. Mastering text-to-image diffusion: Recaptioning, planning, and generating with multi-modal llms. In *International Conference on Machine Learning*, 2024. [6](#), [12](#)
- [39] Zhengyuan Yang, Jianfeng Wang, Zhe Gan, Linjie Li, Kevin Lin, Chenfei Wu, Nan Duan, Zicheng Liu, Ce Liu, Michael Zeng, and Lijuan Wang. Reco: Region-controlled text-to-image generation. In *CVPR*, 2023. [2](#)
- [40] Oz Zafar, Lior Wolf, and Idan Schwartz. Iterative object count optimization for text-to-image diffusion models, 2024. [2](#), [6](#), [12](#)
- [41] Ruisu Zhang, Yicong Chen, and Kangwook Lee. Zero-shot improvement of object counting with CLIP. In *R0-FoMo: Robustness of Few-shot and Zero-shot Learning in Large Foundation Models*, 2023. [2](#)
- [42] Dewei Zhou, You Li, Fan Ma, Xiaoting Zhang, and Yi Yang. Migc: Multi-instance generation controller for text-to-image synthesis, 2024. [14](#)